Do Generate–Verify–Repair Guards Improve LLM NetOps Outputs? A Paired Emulator Study

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

***Abstract*—We preregistered and executed a paired confirmatory experiment (n=200 natural-language NetOps intents; study_id cnd-001-5f3a) that compares ungated LLM generation to a bounded generate–verify–repair (GVR) pipeline applied to a single shared LLM candidate per intent. Generation used gpt-5-mini and the guarded arm applied a deterministic three-stage validator and a deterministic, rule-based repair operator with repair_budget <=1 (no extra LLM calls). Artifact-backed primary outcomes: baseline successes = 199/200 (0.995), guarded = 200/200 (1.000); paired contingency n11=199, n10=1, n01=0, n00=0; paired difference (guarded - baseline) = 0.005. The preregistered exact two-sided McNemar (exact binomial on discordant pairs) yields p = 1.0; paired-bootstrap (10,000 resamples, seed 1729) 95% percentile CI = [0.0, 0.015]. The preregistered 0.15 absolute-effect target is excluded by the observed CI because the realized baseline was near-ceiling. We provide concise, auditable descriptions of the validator and repair operator, execution accounting (model_calls_used = 201; deterministic repair invocations = 1), exact-test justification appropriate for small discordant counts, and operationally grounded limitations. All prompts, provider traces, validator traces, repair traces (the single invoked repair trace), and analysis artifacts are archived in the study evidence bundle.**

I. Introduction

Automating routine network-operations (NetOps) changes with large language models (LLMs) promises reduced manual effort but raises an operational-safety question: do LLM-produced configuration artifacts execute correctly when applied to control planes? Prior work demonstrates intent-to-configuration prototypes and documents structured-output failures (missing commands, misordering, protected-field modification) and proposes mitigations such as retrieval-augmentation, constrained decoding, verifier-assisted repair, and multi-stage tool-augmentation. Despite these proposals, there is limited execution-grounded evidence that quantifies the marginal benefit of applying a bounded deterministic repair to the same initial LLM candidate (a low-hosted-call operational estimand).

We preregistered and executed a paired experiment (study_id cnd-001-5f3a) that isolates the "repair-on-same-candidate" question by sharing a single initial LLM candidate across both arms (baseline ungated acceptance and guarded generate–verify–repair). The shared-candidate estimand is operationally relevant when hosted-model call budgets are constrained. Our contributions are: (1) a preregistered, artifact-backed paired evaluation under the shared-candidate estimand; (2) concise algorithmic descriptions of the deterministic three-stage validator and the deterministic, rule-based repair operator used in the guarded pipeline; and (3) complete execution accounting and exact-test justification for small-discordant-pair inference.

II. Related Work

Research exploring LLMs for NetOps has produced three interlocking strands that motivate the present study. First, intent-to-configuration work demonstrates that LLMs can synthesize device CLI-like commands from natural-language intents but that structured-output errors are common without additional constraints (e.g., missing required commands, incorrect ordering, or violation of protected-field policies). Empirical intent-translation prototypes and surveys document these failure modes and report prototype success under curated conditions (Nguyen et al., 2024) [doi:10.1002/nem.2313]. Complementary surveys and architectural analyses (Bilal et al., 2026) characterize the operator-facing requirements—auditability, bounded tool use, and verifier integration—needed for operational adoption.

Second, mitigation approaches have emerged from both applied and adjacent domains. Retrieval-augmented generation (RAG), live schema grounding, and constrained or grammar-aware decoding have reduced hallucination in tasks such as NL2SQL and document-grounded generation; grammar/constrained decoding methods (e.g., Geng et al., 2023) demonstrate that enforcing syntactic or schema constraints at decode-time reduces format errors without model fine-tuning. Clinical and document RAG studies further show that grounding plus deterministic validators reduces high-stakes hallucinations (Wołk, 2025) [analysis in evidence bundle], but these techniques require careful adaptation for device-specific CLI dialects and workflow policies.

Third, deterministic formal verification and emulator-based validators are natural assurance layers. Formal-methods variants (NetKAT/weighted NetKAT and ensemble verifiers) provide provable checks for reachability and isolation properties (Suárez Acevedo et al., 2026) [doi:10.1145/3808318], and emulator-backed validation captures many syntactic and stateful correctness conditions. Prior work highlights, however, that verification has limits: control-plane dynamics (e.g., BGP convergence timing), vendor idiosyncrasies, and runtime transient rules are not always captured by static or deterministic checks.

Generate–verify–repair (GVR) architectures—where generation is followed by deterministic or model-assisted checking and then repair—are proposed as pragmatic operational patterns, but fully instrumented, execution-grounded evaluations

remain scarce. Surveys and position papers call for workflow-centered benchmarks that include per-intent execution outcomes and repair traces; the literature documents proposals and partial evaluations but lacks preregistered paired measurements that isolate repair-on-the-same-candidate under a constrained model-call budget (the gap G2 noted in our evidence synthesis). Finally, adversarial-prompt research (Das et al., 2026) underscores a persistent operational risk: unconstrained LLM outputs remain attackable, increasing the imperative for auditable deterministic validators in NetOps pipelines. Our study situates itself explicitly within this empirical gap by measuring the marginal, auditable yield of a deterministic repair operator applied only to the shared initial candidate and by using an emulator+deterministic validator as the execution-grounded scoring mechanism.

III. Methodology

Preregistration and high-level design: We preregistered a paired, shared-initial-generation confirmatory protocol (study_id cnd-001-5f3a). For each of n = 200 curated natural-language intents the hosted model produced one initial candidate configuration which was then scored directly as the baseline arm and also passed to the guarded arm. The guarded arm applied a deterministic validator; when the validator reported a repairable violation the pipeline invoked a deterministic, rule-based repair operator (repair_budget <= 1 for the confirmatory run). The primary estimand is the paired success-rate difference (guarded - baseline); the preregistered primary test is the exact two-sided McNemar implementation (exact-binomial on discordant pairs). Planned maximum hosted-model calls = 400; realized model_calls_used = 201 (shared_initial_generation = 200; deterministic repair-stage calls = 1). The execution environment combined Dockerized Mininet+FRR with deterministic_netops_validator_v5 as the canonical validator.

Task bank and operational scoring: The confirmatory bank contains 200 intent–expected-configuration pairs created by a deterministic task generator. Each task payload includes initial_state, expected_final_state, required_sequence (ordered per-task commands the authoritatively required), workflow policies (e.g., make-before-break, do-not-touch protected interfaces), and difficulty metadata. The scoring rule (full_intent_constraint_validation) requires that generated configurations (a) parse syntactically into canonical CLI-like statements, (b) include the task's required_sequence (order and presence), (c) respect dependency and group-activity policies (e.g., minimum-active constraints), (d) preserve protected-interface values, and (e) produce an emulator-observed final_state equal to expected_final_state when applied in the deterministic emulator. Provider/API generation failures are treated per the preregistered missingness plan as arm failures; none occurred in the confirmatory run.

Deterministic validator: The validator deterministically executes three ordered stages and emits a machine-readable trace that the scoring and repair logic consume. Stage 1 – syntactic parsing and canonical normalization: tokenizes CLI-like lines, canonicalizes variant synonyms, and produces normalized_configuration; syntax failures yield a SYNTAX_PARSE_ERROR and block further deterministic repair. Stage 2 – structural and workflow checks: the validator deterministically verifies presence and ordering of required_sequence entries and enforces dependency rules and group policies; it emits deterministic violation codes such as MISSING_REQUIRED_COMMAND, DEPENDENCY_VIOLATION, PROTECTED_INTERFACE_MODIFICATION, and TRANSIENT_ORDER_VIOLATION. Stage 3 – deterministic emulator application: the normalized_configuration is applied to the emulator to produce observed final_state, and the validator reports validation_after.valid based on exact equality with expected_final_state. The validator trace fields archived per episode (and used by analysis) include parsed_command_count, observed_touched_field_count, normalized_configuration, violation_count, violation_codes, validator_id and validator_version, and validation_before/after summaries. These artifacts support auditable, post-hoc inspection of every failure mode.

Repair operator: The preregistered repair operator is a deterministic, rule-based transformer invoked only when the validator reports violation_count > 0 and the violation family is in a preregistered repairable set. The operator enforces the post-lock constraints: (i) no additional LLM calls (repairs are implemented by deterministic code); (ii) bounded budget (<=1 repair attempt per episode in confirmatory runs); and (iii) unambiguous mapping requirement (repairs are applied only when the mapping from violation code to transformation is unique given the task payload). Repair decisions are computed from the task manifest (not inferred by an LLM): the operator consults required_sequence, initial_state, expected_final_state, and workflow_policies to deterministically generate corrections. The operator implements three canonical deterministic transformations: - Insertion: when a MISSING_REQUIRED_COMMAND violation exists and the missing step appears in the task's required_sequence, the operator inserts the missing command(s) at canonical positions defined by the required_sequence. Positioning rules are deterministic: insert before the first command that references the same interface/field group or at the canonical step index derived from the required_sequence ordering in the manifest. - Reordering: for DEPENDENCY_VIOLATION or TRANSIENT_ORDER_VIOLATION the operator performs a stable reordering of contiguous, annotated command blocks to satisfy dependency constraints while preserving intra-block lexical order. The reordering algorithm deterministically identifies minimal block moves that resolve the dependency according to the manifest's dependency rules. - Preservation enforcement (redaction/restore): for PROTECTED_INTERFACE_MODIFICATION or when preserve_unspecified_state would be violated, the operator deterministically replaces the offending command(s) with the original value obtained from initial_state (or, if required by workflow_policies, with the canonical preserved value from

expected_final_state).

Ambiguity and nonrepairable cases: If a violation admits multiple equally plausible deterministic repairs (ambiguous insertion indices, multiple candidate values), the operator abstains (no change) and the episode remains a guarded failure; ambiguous families and policy-contradictory violations are flagged in the repair_trace and counted in analysis as nonrepairable. All repair decisions and the resulting normalized_configuration are recorded in a repair trace archived with the run artifacts. In the confirmatory run the deterministic repair operator was invoked exactly once and that single repair converted its episode to a validator-passing configuration.

Statistical analysis: The primary confirmatory test is McNemar's paired binary test implemented in its exact-binomial (exact McNemar) form appropriate for small discordant counts. Let n10 be the number of pairs where guarded = 1 and baseline = 0, and n01 where baseline = 1 and guarded = 0; n = n10 + n01 is the total number of discordant pairs. The exact two-sided p-value is computed from the binomial distribution on n trials assuming p = 0.5 (symmetry under the null): the two-sided p equals twice the smaller of the lower and upper tail probabilities for observing k = n10 successes under Binomial(n, 0.5), truncated at 1.0. For our observed counts (n10 = 1, n01 = 0, n = 1) this exact-binomial two-sided computation yields p = 1.0. Complementary uncertainty quantification uses a paired-bootstrap percentile 95% CI for the guarded - baseline proportion difference with 10,000 resamples (bootstrap_seed = 1729) as preregistered; these parameters and the bootstrap result are archived in analysis artifacts. The preregistered power analysis assumed a baseline $\approx$0.5 and targeted a 15 percentage-point absolute effect at n = 200; the realized near-ceiling baseline substantially reduces the expected discordant counts and consequent sensitivity for that target effect size. All analysis scripts, seeds, resample counts, and the paired contingency CSV are archived in the evidence bundle for reproducibility.

Operational accounting and failure rules: Provider/API transient errors were treated per the preregistered missingness plan (single automatic retry for transient timeouts with 60s backoff); no such retries were consumed in the confirmatory run. Episodes failing syntactic parse are unrepaired by design (repair operator requires a valid normalized_configuration); such parse failures were zero in the confirmatory bank. Episodes where repair abstains are reported as guarded failures and counted in the paired contingency table. The deterministic pipeline's audit traces (per-episode validator traces and any repair traces) preserve the exact sequence of decisions for post-hoc inspection and are referenced by analysis outputs.

IV. Results

Execution and primary outcomes: The confirmatory run completed all planned episodes (400 episodes = 200 pairs) with no provider/API failures. Artifact-backed primary counts: baseline_success_count = 199/200 (0.995) and guarded_success_count = 200/200 (1.0). The paired contingency table is n11 = 199, n10 = 1 (guarded-only), n01 = 0 (baseline-only), n00 = 0; paired difference (guarded - baseline) = 0.005. The preregistered exact two-sided McNemar (exact binomial on discordant pairs) yields p = 1.0 for the observed ordering (n = 1). The paired-bootstrap (10,000 resamples, seed 1729) 95% percentile CI is [0.0, 0.015]. All per-episode JSONL scoring artifacts and the analysis result JSON are archived.

Repair and failure accounting: validation_before.valid == false occurred in exactly one confirmatory episode; deterministic repair invocations = 1 and that single repair converted the failing candidate into a validator-passing final configuration, producing the sole guarded-only success (n10 = 1). Syntactic/parse failures across the confirmatory set = 0. Provider-call audit: hosted_model_call_event_count = 201, completed_hosted_model_call_event_count = 201, cache_miss_count = 201, stage_counts show shared_initial_generation = 200 and repair = 1. No episodes were excluded for contamination; missingness summary reports complete_pairs = 200 and zero unscorable episodes.

Representative exemplars and difficulty context: we archive six representative exemplars with per-example difficulty labels (medium = 1, hard = 2, very_hard = 3) to illustrate typical intent patterns including shutdown_configure_restore, make-before-break uplink switchover, and paired atomic MTU changes. The full per-episode difficulty distribution for all 200 confirmatory intents is recorded in the archived task manifest (execution/task_manifest.jsonl) and is available for re-analysis; because aggregated counts are stored in that manifest we reference it for exhaustive difficulty counts rather than enumerating the full distribution inline.

Sensitivity and interpretive statistics: The preregistered 0.15 absolute-effect target was chosen under a baseline $\approx$0.5 assumption and is excluded by the observed bootstrap CI; the near-ceiling baseline compresses attainable discordant counts and reduces power to detect large absolute improvements under the shared-candidate estimand. Practically, deterministic repair-on-same-candidate produced a single marginal improvement in this curated confirmatory bank.

V. Discussion

Operational interpretation: Under the shared-initial-generation estimand and the curated confirmatory distribution the ungated gpt-5-mini generator produced near-perfect candidates (199/200). Deterministic, auditable validators produce structured violation codes that enable targeted deterministic repairs; in low-error regimes the marginal yield of a conservative rule-based repair on the same candidate is small but auditable and low-cost. For operators constrained by model-call budgets, the guarded shared-candidate design preserves low hosted-call cost while providing an assurance gate; for contexts prioritizing maximal success, independent resampling or LLM-invoked repair loops merit exploration.

Design-space trade-offs and practitioner guidance: The confirmatory run's execution accounting (model_calls_used = 201; repair invocations = 1; completed_episodes = 400) enables simple cost-versus-yield comparisons. If operators accept

higher hosted-call expense, independent-resampling (two or more independent LLM generations per intent) or LLM-guided iterative repair could raise success rates at the cost of greater provider usage and more complex governance. Deterministic validators should be considered mandatory assurance machinery where auditable, repeatable checks and low-latency failure explanations are required.

Threats to validity: Internal validity is strong for the paired estimand and deterministic scoring rules; construct validity depends on emulator fidelity—our deterministic emulator models control-plane state deterministically but cannot fully capture runtime effects such as BGP convergence timing or specific device firmware idiosyncrasies. External validity is limited by the single LLM (gpt-5-mini) and a synthetic curated bank focused on small changes; extrapolation to multivendor CLI diversity or adversarial/out-of-distribution intents is limited. Conclusion validity is constrained by the near-ceiling baseline that reduced discordant-pair counts; the exact McNemar is the correct small-sample test for this observed pattern but offers limited sensitivity when n is small. These limits are documented in the preregistration and archived manifests.

Reproducibility and auditability: All artifacts needed to reproduce analysis and inspection are archived in the evidence bundle: per-episode prompts and responses, provider-call traces, validator traces, deterministic repair trace (the single invoked repair), task manifest entries, analysis scripts, paired contingency CSV, bootstrap parameters (seed = 1729; resamples = 10,000), and execution manifest (preregistration_sha256: e5c5fdbd717c...). The exact-test implementation is the exact-binomial McNemar formulation applied to discordant-pair counts (n10,n01) and is described above; the analysis log records the p-value and bootstrap CI used in the manuscript.

VI. Limitations

- Synthetic, curated confirmatory task bank focused on small single- and multi-device changes; not comprehensive of production NetOps workloads (multivendor CLI dialects, hardware idiosyncrasies, dynamic control-plane timing).
- Single LLM (gpt-5-mini) evaluated; findings do not imply multi-model generality.
- Deterministic emulator + validator model control-plane behavior but cannot fully capture real-world runtime dynamics such as BGP convergence timing or vendor-specific runtime effects.
- Preregistered repair budget limited to a single deterministic repair iteration; different repair strategies or additional iterations might yield different outcomes.
- Shared-initial-generation design reduces model-call cost and isolates repair-on-same-candidate effectiveness but reduces external generalizability compared with independent-resampling estimands.
- Near-ceiling baseline success limited statistical power to analyze failure-mode distributions in the confirmatory set; exploratory failure-taxonomy conclusions are therefore constrained.
- Adversarial or out-of-distribution intents (e.g., jailbreaks, malformed ambiguous intents) were not included in the confirmatory set and require separate evaluation.
- Full per-task difficulty label counts for all 200 confirmatory intents are recorded in execution/task_manifest.jsonl in the archived evidence bundle; the manuscript references that manifest rather than enumerating the full 200-line distribution inline to preserve concise presentation while preserving auditable reproducibility.

VII. Conclusion

We executed an autonomy-locked, preregistered paired experiment to measure whether a bounded generate–verify–repair guard improves emulator-executable success for LLM-generated NetOps configurations under a shared-candidate generation budget. Ungated gpt-5-mini generation succeeded in 199/200 confirmatory cases; the deterministic guarded pipeline succeeded in 200/200. The observed paired difference = 0.005 (exact McNemar two-sided p = 1.0; paired-bootstrap 95% CI [0.0, 0.015]) does not support the preregistered 0.15 absolute-effect target because the baseline was near-ceiling. For this estimand and task distribution, deterministic repair-on-same-candidate yielded negligible marginal improvement, but deterministic validators remain valuable, auditable assurance gates for operational NetOps pipelines. Full artifacts and analysis code are archived with the study for independent audit and re-analysis.

Disclosure Statement

This study was executed autonomously after an autonomy lock defined by the initial master prompt archived at provenance/master_prompt.txt (SHA-256: 1872df1e1805d2d9 6940456ca016bd665d1d5196add77f5acdf1582bb39b15ba). Human involvement prior to the lock was limited to selecting the topic family (Generative AI and LLMs for NetOps) and approving the master prompt. No manual human editing of technical content, figures, captions, references, results, or manuscript text occurred after the lock. The autonomous pipeline performed literature search and verification, research-gap selection, preregistration (study_id cnd-001-5f3a), code generation, experiment execution, artifact capture, analysis, internal critique, peer-review checks, and manuscript drafting.

Models and tooling: generation requested gpt-5-mini (declared_model_version in execution/model_configuration.json). Validation used deterministic_netops_validator_v5 implementing syntactic parsing, structural/workflow checks, and deterministic emulator application. Repairs in the confirmatory run were applied by deterministic code only (no additional LLM calls). The execution environment used Dockerized Mininet+FRR images orchestrated by adapter family hosted_netops_gvr_v1. Preregistered and enforced settings (not altered post-lock) include: primary estimand = paired_success_rate_difference_guarded_minus_baseline; confirmatory sample = n=200 paired intents; maximum

model-call budget = 400; repair budget <=1 deterministic repair iteration; primary analysis = exact McNemar two-sided (exact-binomial on discordant pairs); bootstrap CIs = 10,000 resamples, seed = 1729. Execution accounting (artifact-backed): the run completed 400 episodes with model_calls_used = 201 (shared_initial_generation = 200; repair-stage calls = 1). All prompts, provider-call traces, responses, validator traces, repair traces (the single invoked repair trace), analysis artifacts, and the execution manifest are archived in the study evidence bundle.

REFERENCES

[1] Nguyen Tu, Sukhyun Nam, James Won-Ki Hong, "Intent-Based Network Configuration Using Large Language Models", 2024, 10.1002/nem.2313

[2] Emmanuel Suárez Acevedo, Tiago Ferreira, Kevin Batz, Oliver Bøving, Nate Foster, Alexandra Silva, "Weighted NetKAT: A Programming Language for Quantitative Network Verification", 2026, 10.1145/3808318

[3] Sanjay Mishra, Divya Chukkapalli, Ganesh R. Naik, "Schema-Aware Localisation (SAL): Live Schema Grounding and Hallucination Validation for Oracle NL2SQL", 2026, 10.2139/ssrn.6909831

[4] Muhammad Bilal, Jon Crowcroft, Ruizhi Wang, Xiaolong Xu, Schahram Dustdar, "Large Language Models for Agentic NetOps and AIOps: Architectures, Evaluation, and Safety", 2026, 10.48550/arxiv.2605.12729

[5] Nilanjana Das, Edward Raff, Aman Chadha, Manas Gaur, "Human-Readable Adversarial Prompts: An Investigation into LLM Vulnerabilities Using Situational Context", 2026, 10.1145/3815174